CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 2 Divide-and-Conquer

ملخص الميد · Insertion Sort · Merge Sort

محارب

محتوى الملف

01 الفكرة

02 Insertion Sort

03 Loop Invariant

04 Merge Sort

05 تحليل Merge Sort

06 n log n vs n²

07 Merge Sort vs Quick Sort

08 ورقة المراجعة

01 الفكرة

Divide and conquer طريقة لتصميم خوارزميات سريعة: نقسم المشكلة الكبيرة إلى مشاكل أصغر من نفس النوع، نحل الصغيرة، وبعدين نجمع الحلول.

#	الخطوة	وش تسوي
1	**Divide**	نقسم المشكلة إلى sub-problems من نفس شكل الأصلية.
2	**Conquer**	نحل كل sub-problem لحالها، وغالباً بالـ recursion.
3	**Combine**	نجمع حلول الـ sub-problems عشان نطلع حل المشكلة الأصلية.

المثالين الكلاسيكيين: **Merge sort** و **Quicksort**.

وش يجي منها في الميد

جملة True/False عن معادلة وقت **MERGESORT** تكررت في ميدين. وفي الميد يجي MCQ يفرّق بين Divide and conquer و Dynamic programming و Greedy.

Insertion Sort 02

ندخّل العناصر وحد وحد. نبدأ من العنصر الثاني، ونسحبه لورا لين يوصل مكانه الصحيح.

```
InsertionSort(A)
   for i ← 2 to length(A) do
      key ← A[i]
      j ← i − 1
      while j > 0 and A[j] > key do
         A[j + 1] ← A[j]
         j ← j − 1
      A[j + 1] ← key
```

مثال المحاضرة على `6 4 3 8 5` :

i	key	Before	After
2	4	6 4 3 8 5	4 6 3 8 5
3	3	4 6 3 8 5	3 4 6 8 5
4	8	3 4 6 8 5	3 4 6 8 5
5	5	3 4 6 8 5	3 4 5 6 8

الحالة	المدخل	الوقت
Best case	مرتب أصلاً. الـ while ما تلف.	$O(n)$
Worst case	مرتب بالعكس. الـ while تلف لين البداية كل مرة.	$O(n^2)$

ليش n^2؟ اللوب الخارجي يلف $n - 1$ مرة، والداخلي في أسوأ حالة يلف حوالي n مرة.

Loop Invariant 03

الـ **loop invariant** شي لازم يكون صحيح في كل لفة. نستخدمه عشان نثبت أن الخوارزمية صحيحة. في Insertion sort: «قبل اللفة `i` ، أول `i - 1` عنصر مرتبين».

الخطوة	معناها
Initialization	الـ invariant صحيح قبل أول لفة.
Maintenance	لو صحيح قبل اللفة `t` يبقى صحيح قبل اللفة `t + 1` .
Termination	صحيح بعد آخر لفة، وهذا اللي يعطينا النتيجة: المصفوفة كلها مرتبة.

InsertionSort is an algorithm that correctly sorts an arbitrary n-element array in time about n^2.

Merge Sort 04

```
MERGESORT(A):
    n ← length(A)
    if n ≤ 1:
        return A                        already sorted
    L ← MERGESORT(A[1 : n/2])           sort the left half
    R ← MERGESORT(A[n/2 + 1 : n])       sort the right half
    return MERGE(L, R)                  merge the two halves
```

مثال المحاضرة على `6 4 3 8 1 5 2 7` :

```
Divide     [6 4 3 8 1 5 2 7]
           [6 4 3 8]          [1 5 2 7]
           [6 4]   [3 8]      [1 5]   [2 7]
           [6] [4] [3] [8]    [1] [5] [2] [7]
Merge      [4 6]   [3 8]      [1 5]   [2 7]
           [3 4 6 8]          [1 2 5 7]
           [1 2 3 4 5 6 7 8]
```

الـ **MERGE** ياخذ مصفوفتين مرتبتين ويمشي عليهم بمؤشرين: ياخذ الأصغر من الراسين كل مرة. وقته خطي في مجموع الطولين.

ليش تشتغل؟ بالـ **recursion invariant**: «في كل recursive call، **MERGESORT** ترجع مصفوفة مرتبة».

الخطوة	في Merge sort
Base case	`n = 1`: مصفوفة من عنصر واحد مرتبة دايماً.
Maintenance	لو `L` و `R` مرتبين، فـ `MERGE(L, R)` مرتب.
Termination	أعلى call ترجع المصفوفة كلها مرتبة.

Merge sort runs in O(n log n) time in all the cases.

05 تحليل Merge Sort

```
T(n) = T(n/2) + T(n/2) + T(MERGE)
```

```
T(n) = time to run MERGESORT on a list of size n
```

يعني: وقت ترتيب النص اليسار + وقت ترتيب النص اليمين + وقت الدمج.

جملة الميد

> The time to run MERGESORT on a list of size n, T(n) = T(n/2) + T(n/2) + T(MERGE(n/2))

الميد غيّر آخر حد: في 472 كتبه `T(MERGE(n/2))` وفي 451 كتبه `T(MERGE/2)` . السلايد يكتبه `T(MERGE)` بس، والدمج يشتغل على `n` عنصر (نصين كل واحد `n/2`). فالأقرب أن الجملة **False** بالصيغتين. ما عندي نموذج إجابة رسمي، فتأكد من الدكتور.

الـ **Recursion tree**: نرسم كل recursive call كعقدة ونحسب الشغل في كل مستوى.

Level	# problems	Size of each	Work at this level
0	1	n	6n
1	2	n/2	6n
2	4	n/4	6n
t	2^t	$n/2^t$	6n
log(n)	n	1	6n

```
Work per level   =  2^t problems × 6(n/2^t)  =  6n
```

```
Number of levels =  log(n) + 1
```

```
Total            =  6n(log(n) + 1)   →   O(n log n)
```

الـ `6n` هو شغل الدمج في المستوى كله. الرقم 6 ثابت من عدّ عمليات **MERGE**، والمهم أنه خطي.

06 n log n vs n²

`log(n)` يعني: كم مرة نقسم `n` على 2 لين نوصل 1؟ كل الـ logarithms في المادة **base 2**.

```
32 → 16 → 8 → 4 → 2 → 1          log(32) = 5
```

```
log(64) = 6     log(128) = 7     log(256) = 8     log(512) = 9
```

n	n log(n)	n²
8	24	64
16	64	256
32	160	1024
64	384	4096
128	896	16384
256	2048	65536
512	4608	262144
1024	10240	1048576

`log(n)` يكبر ببطء شديد، عشان كذا `n log n` أحسن بكثير من `n²` لما `n` يكبر.

07 Merge Sort vs Quick Sort

الاثنين divide and conquer، والاثنين recursive، والاثنين يرتبون بالمقارنة. الفرق:

	Merge sort	Quick sort
التقسيم	من النص دايماً، نصين متساويين.	حول **pivot**، وممكن ما ينقسم بالتساوي.
مصفوفة إضافية	يحتاج.	ما يحتاج.
خطوة الدمج	يحتاج.	ما يحتاج.
Best / Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n \log n)$	$O(n^2)$

فكرة **Quick sort**: نختار pivot، نحط الأصغر منه يساره والأكبر يمينه (partition)، وبعدين نرتب كل جهة بنفس الطريقة.

08 ورقة المراجعة

السؤال	الجواب
خطوات divide and conquer؟	Divide → Conquer → Combine
Insertion sort	Best O(n) · Worst $O(n^2)$
Merge sort	O(n log n) in all cases
معادلة Merge sort؟	T(n) = T(n/2) + T(n/2) + T(MERGE)
عدد مستويات الشجرة؟	log(n) + 1
المستوى `t` ؟	2^t problems of size $n/2^t$
خطوات الـ loop invariant؟	Initialization · Maintenance · Termination
أسوأ حالة Quick sort؟	$O(n^2)$